

Scaling Laws for Transformer Language Models Trained on SVG Code

CS-GY 6923 – Optional Project

Disha D Shanbhag
NetID: ds8223 N11461466
NYU Tandon School of Engineering

April 30, 2026

Abstract

This project investigates neural scaling laws for decoder-only transformer language models trained to generate Scalable Vector Graphics (SVG) code. Using over 115 million tokens of structured SVG data drawn from three HuggingFace datasets, models ranging from 1.35 million to 91.5 million parameters are trained and their validation losses compared. Under standard parameterization (SP), the validation loss is non-monotonic with scale: the largest model performs worse than a model one quarter its size, revealing that a fixed learning rate is suboptimal across widths. Maximal Update Parameterization (μ P) corrects this behavior, yielding a smooth power-law fit with $\alpha = 0.866$ and $R^2 = 0.960$. The best model (XL, μ P, 5 epochs) achieves a test perplexity of 2.51 and an SVG validity rate of 82.6%. Analysis of generated samples shows the model reliably learns SVG structural conventions while coherent spatial layout emerges only partially at this scale.

1 Introduction

Scaling laws for language models describe how validation loss decreases predictably as a power function of model size, dataset size, and compute [1, 2]. These laws have been empirically verified for natural language but their behavior in structured, non-linguistic domains remains less studied. Scalable Vector Graphics (SVG) provides a compelling test case: it is XML-based text that encodes visual content through strict syntactic rules (valid XML, coordinate systems, color values), hierarchical structure (nested groups and transforms), and instantaneously renderable outputs.

Unlike natural language, SVG has a small effective vocabulary (a few hundred common element types and attributes), short local dependencies within a single element, and long global dependencies that determine visual coherence across elements. These properties suggest that scaling laws derived for natural language may not transfer directly to the SVG domain.

This project trains five decoder-only transformer models at scales from 1M to 88M parameters on a corpus of approximately 115 million SVG tokens. It investigates (1) whether power-law scaling holds on SVG data, (2) whether μ P enables principled learning rate transfer across model widths where standard fixed-LR training fails, and (3) what visual and structural patterns emerge at different scales.

2 Data and Preprocessing

2.1 Datasets

Three HuggingFace datasets are combined to form the training corpus:

- **starvector/svg-icons-simple** [4]: 80,434 simplified SVG icons.
- **starvector/svg-emoji-simple**: 4,114 simplified SVG emoji.
- **starvector/svg-fonts-simple**: 200,000 font glyph SVGs (subsampled via streaming to manage size).

These datasets are all simplified outputs from the StarVector pipeline and contain short, clean SVG code. The combined raw corpus contains 284,548 SVGs.

2.2 Cleaning and Normalization

Each SVG passes through a normalization pipeline implemented with Python’s `re` module and `lxml`:

1. Strip HTML comments (`<!--...-->`), metadata tags (`<metadata>`, `<title>`, `<desc>`), and collapse repeated whitespace.
2. Normalize coordinate precision to one decimal place using regex substitution on floating-point literals, reducing vocabulary fragmentation.
3. Validate each SVG as well-formed XML using `lxml.etree.fromstring`.
4. Remove SVGs shorter than 50 characters (none were found in these pre-simplified datasets).

All 284,548 SVGs pass the cleaning step without failure. This is consistent with the pre-simplified nature of the StarVector datasets. CairoSVG render validation was not applied during preprocessing (it was used during generation evaluation in Part 4).

2.3 Tokenization

A Byte-Pair Encoding (BPE) tokenizer is trained on the full SVG corpus using the HuggingFace `tokenizers` library with a byte-level pre-tokenizer. The vocabulary size is set to 4,096 tokens, chosen as the midpoint of the recommended range (1K–8K). This size is large enough to represent common multi-character SVG patterns (e.g., `fill=`, `viewBox=`) as single tokens, reducing sequence lengths, while remaining small enough for the embedding table to fit comfortably in a 1M-parameter model.

Four special tokens are included: `<PAD>`, `<BOS>`, `<EOS>`, and `<UNK>`. Each SVG in the binary training files is wrapped with `<BOS>` and `<EOS>` tokens.

2.4 Train / Validation / Test Splits

After tokenization, SVGs with more than 1,024 tokens are removed to keep context window requirements manageable for the smallest models. Splits are assigned by file index (after global shuffle) to avoid data leakage. Table 1 summarizes the final corpus.

Table 1: Dataset statistics after cleaning and tokenization.

Statistic	Value
Vocabulary size	4,096
Raw SVGs (all sources)	284,548
SVGs after XML validation	284,548
SVGs after token-length filter ($\leq 1,024$)	236,529
Train files	231,799
Train tokens	115,916,260
Validation files	2,365
Validation tokens	1,176,658
Test files	2,365
Test tokens	1,186,563
Sequence length – mean	500
Sequence length – median	475
Sequence length – std	199
Sequence length – min	93
Sequence length – max	1,024
Sequence length – P25 / P75 / P95	354 / 622 / 889

Figure 1 shows the sequence length distribution. The distribution is approximately unimodal and right-skewed, with the bulk of sequences between 300 and 700 tokens, reflecting the moderate complexity of the icon-style SVGs in the corpus.

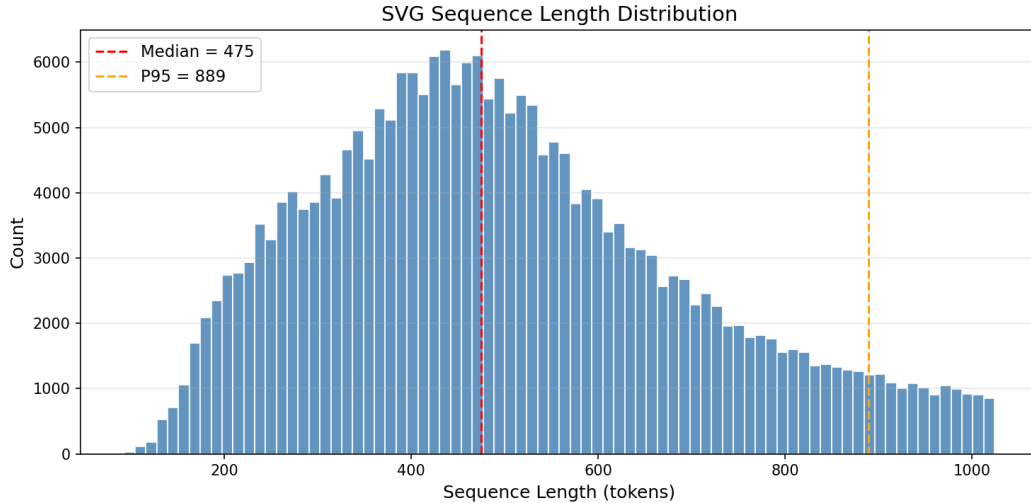


Figure 1: Sequence length distribution over 236,529 filtered SVGs. The red dashed line marks the median (475 tokens); the orange dashed line marks the 95th percentile (889 tokens). The context window is capped at 1,024 tokens.

2.5 SVG Complexity Examples

To illustrate the range of visual complexity in the corpus, Listing 1 shows a representative short SVG (5th percentile, 211 tokens). This icon-style glyph consists of a single `<path>` element with explicit stroke and fill settings and a normalized `viewBox`. At the 95th percentile (889 tokens), SVGs contain multiple parallel path strokes forming glyphs such as the letter “A”, with dozens of coordinate pairs per path.

Listing 1: Example SVG at the 5th percentile of sequence length (211 tokens after BPE). Coordinates are rounded to one decimal place by the normalization pipeline.

```
1 <svg xmlns="http://www.w3.org/2000/svg"
2   viewBox="0.0 0.0 24.0 24.0"
3   height="200px" width="200px">
4   <path fill="none" stroke="black"
5       stroke-width=".3" stroke-opacity="1.0"
6       d="M16.5 7.8 L14.9 15.0 ..."/>
7 </svg>
```

3 Methods

3.1 Model Architecture

All models are decoder-only transformers following the nanoGPT architecture [5]. The `GPT` and `CausalSelfAttention` classes are adapted directly from nanoGPT, retaining its pre-norm design, weight-tied embeddings, and cosine LR schedule. Three modifications are introduced for this project: (1) vocabulary size is set to 4,096 to match the SVG BPE tokenizer; (2) context window is set to 256 tokens (down from 1,024) to accommodate the full model size range within A100 memory; and (3) μ P variants replace standard attention scaling and the output head (Section 3.4).

Key design choices:

- Pre-norm (LayerNorm before attention and MLP).
- Causal self-attention with a learned causal mask.
- GELU activation in the feed-forward network.
- Learned absolute positional embeddings.
- Weight tying between the token embedding table and the output projection (SP only).
- Dropout of 0.1 applied to attention weights and residual connections.

The context window is 256 tokens (block size). Although sequences can be up to 1,024 tokens, the 256-token block size was chosen to allow all five model sizes to train within the available A100-SXM4 GPU memory while keeping the batch token budget high.

Five model configurations spanning approximately one decade of parameter counts are trained. Table 2 lists the architectures.

Table 2: Model configurations and training statistics under standard parameterization (SP).

Name	Params	d_{model}	n_{layers}	n_{heads}	d_{ff}	Val Loss	Time (s)	Tok/s
Tiny	1.35M	128	4	4	512	1.229	200	580,939
Small	3.51M	192	6	6	768	1.051	413	282,007
Medium	12.32M	384	6	6	1536	0.947	817	142,500
Large	33.75M	512	10	8	2048	1.067	1951	59,643
XL	88.40M	768	12	12	3072	1.367	4492	25,906

3.2 Training Setup

All models use a consistent setup across Parts 2 and 3:

- **Optimizer:** AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1.
- **Learning rate schedule:** Cosine annealing with linear warmup over 5% of total steps. Minimum LR is set to $0.1 \times \text{max LR}$.
- **Gradient clipping:** Norm clipping at 1.0.
- **Batch size:** 64 sequences $\times$ 256 tokens = 16,384 tokens per step.
- **Steps per epoch:** 7,103 steps over 115M+ tokens.
- **Hardware:** Single NVIDIA A100-SXM4-40GB GPU (42.4 GB VRAM).

The training objective is the standard autoregressive cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}) \quad (1)$$

where T is the sequence length and p_{θ} denotes the model distribution. Perplexity is defined as $\text{PPL} = \exp(\mathcal{L}_{\text{CE}})$.

3.3 Standard Parameterization Scaling Study

A learning rate sweep over 7 values on a log scale (10^{-5} to 10^{-2}) is conducted on the Tiny model. The optimal learning rate is selected by validation loss and then applied to all larger models. Each model is trained for exactly one epoch.

3.4 μP Scaling Study

Maximal Update Parameterization (μP) [3] is implemented using the `mup` Python package. Three key modifications distinguish the μP model from SP:

1. Attention scale changed from $1/\sqrt{d_{\text{head}}}$ to $1/d_{\text{head}}$.
2. Output projection replaced by `MuReadout`, which applies a width-ratio scaling factor to the logits.
3. Optimizer replaced by `MuAdamW`, which assigns per-layer learning rate multipliers derived from the ratio of each layer’s width to the base width.

The base width is set to $d_{\text{model}} = 128$ (the Tiny model width). Base and delta models are constructed using the same depth but widths of 128 and 256, respectively, and passed to `set_base_shapes`. Note that μP models do not support weight tying between embeddings and the output head, resulting in slightly higher parameter counts than their SP equivalents.

3.5 Evaluation Metrics

For generated SVG samples, the following quantitative metrics are computed:

$$\text{ValidityRate} = \frac{\text{XML-valid samples}}{\text{Total samples}} \times 100 \quad (2)$$

$$\text{RenderRate} = \frac{\text{Renderable samples}}{\text{Total samples}} \times 100 \quad (3)$$

XML validity is checked with `lxml.etree`. Two render criteria are used:

- **Strict** (`can_render`): CairoSVG must succeed and the output PNG must contain at least one non-transparent pixel (alpha channel > 0). This rejects blank-canvas outputs.
- **Loose** (formal evaluation): CairoSVG must succeed and return a non-empty PNG byte string, regardless of pixel content.

The loose check is used for the headline validity rate; strict results are reported in the per-temperature breakdown where mode-collapse detection is relevant.

4 Scaling Law Experiments

4.1 Learning Rate Sweep

The Tiny model (1.35M parameters) is trained for one epoch at each of 7 learning rates on a log scale. Table 3 and Figure 2 summarize the results. The validation loss decreases monotonically as the learning rate increases from 10^{-5} to 10^{-2} , with no sign of instability at the highest value tested. The optimal learning rate is $\eta^* = 10^{-2}$, applied to all models in the standard parameterization study.

Table 3: Learning rate sweep on the Tiny model (1 epoch). Best entry marked with *.

Learning Rate	Val Loss	Train Loss
1×10^{-5}	3.058	3.111
3×10^{-5}	2.245	2.229
1×10^{-4}	1.856	1.923
3×10^{-4}	1.525	1.593
1×10^{-3}	1.327	1.359
3×10^{-3}	1.255	1.268
1×10^{-2}	1.221*	1.283

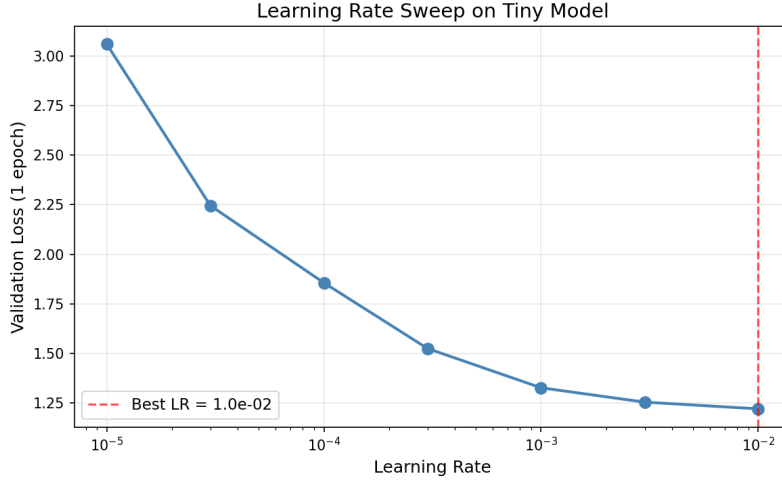


Figure 2: Validation loss as a function of learning rate (log scale) on the Tiny SP model. The red dashed line indicates the selected optimal LR = 10^{-2} .

4.2 Scaling Results and Power Law Fit

The scaling law is formulated as:

$$L(N) = aN^{-\alpha} + c \quad (4)$$

where N is the number of trainable parameters, $L(N)$ is the validation loss after one epoch, $\alpha > 0$ is the scaling exponent, a controls the magnitude of improvement with scale, and c represents an irreducible loss floor. The parameters are estimated via nonlinear least squares using `scipy.optimize.curve_fit`.

Key finding: Under standard parameterization with fixed LR= 10^{-2} , the validation loss is non-monotonic with scale. Medium (12M parameters) achieves the lowest loss (0.947), but Large (33M) and XL (88M) are significantly worse at 1.067 and 1.367 respectively. The power law fit is degenerate ($a \approx 10^6$, $R^2 = 0.041$), reflecting the absence of a consistent scaling signal. This directly demonstrates the motivating problem for μ P: a learning rate optimal for a small model can be too large for a wider model, causing instability or slow convergence.

Figure 3 shows the SP scaling curve. The curve fit is included for completeness but should not be interpreted as a reliable scaling law.

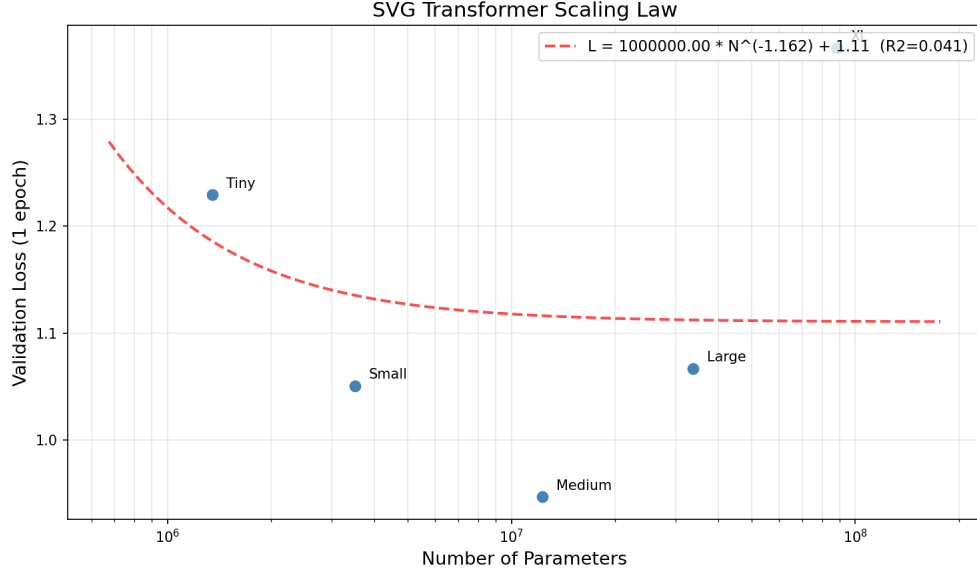


Figure 3: SP scaling curve (validation loss vs. parameters on log scale). The non-monotonic behavior – XL performing worse than both Medium and Small – reveals that the fixed learning rate of 10^{-2} is suboptimal for larger models under standard parameterization. The power law fit ($R^2 = 0.041$) is therefore not predictive.

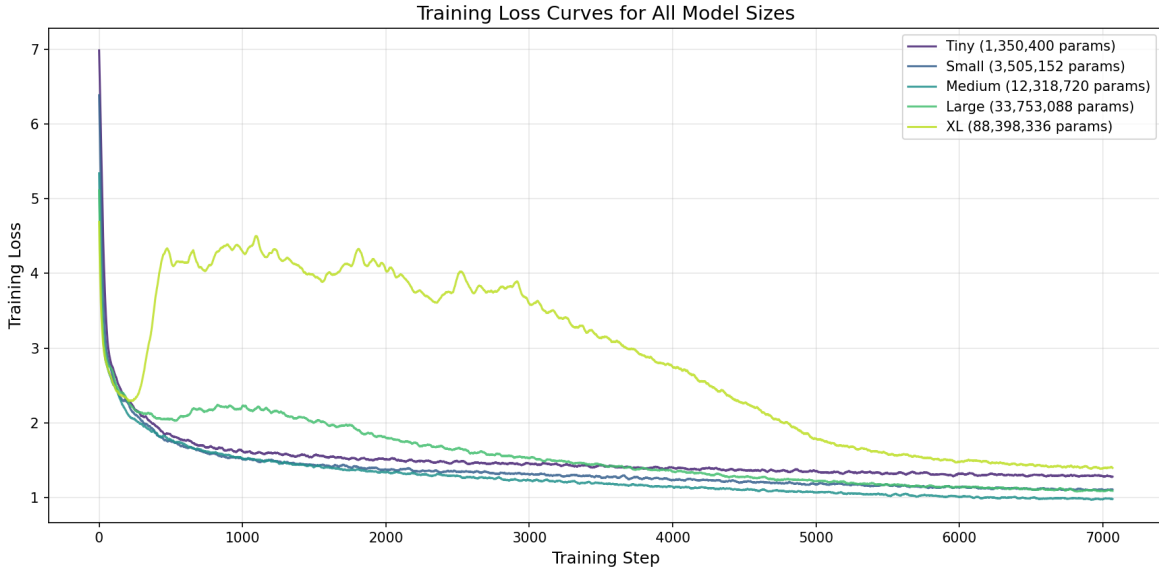


Figure 4: Smoothed training loss curves for all five SP models. Models converge at different rates reflecting their parameter counts and throughput. The XL model shows the slowest convergence in loss-per-step terms and ends at a higher final loss than expected.

Table 4 summarizes wall-clock time and GPU memory usage. Throughput drops approximately proportionally to the increase in parameters, consistent with compute-bound training on a single GPU.

Table 4: Training throughput and resource usage for all SP models (1 epoch).

Model	Wall Time (s)	Tokens/s	Peak GPU (MB)
Tiny	200	580,939	2,286
Small	413	282,007	3,773
Medium	817	142,500	5,152
Large	1,951	59,643	10,136
XL	4,492	25,906	17,687

5 μ P Scaling and Extrapolation

5.1 Background

Under standard parameterization (SP), the optimal learning rate for a small model is generally not optimal for a wider model [3]. Specifically, for overparameterized networks, the feature updates scale as $O(1/d_{\text{model}})$ under SP – meaning wider models receive proportionally smaller updates – which can cause instability or slow convergence at the learning rates selected for narrow models.

μ P [3] addresses this by reparameterizing layer weights and output logits so that the per-parameter update magnitude is independent of width. Under μ P, the optimal learning rate found on a small “proxy” model transfers directly to wider models without retuning, enabling zero-shot hyperparameter transfer.

5.2 Learning Rate Sweep Comparison

Table 5 and Figure 5 compare the LR sweep results for SP and μ P on the Tiny model. Both parameterizations select $\eta^* = 10^{-2}$ as the best learning rate. The validation losses at each LR are very similar between SP and μ P on the Tiny model, as expected – the μ P benefit emerges when the learned LR is transferred to wider models.

Table 5: LR sweep comparison: SP vs. μ P on the Tiny model (1 epoch).

Learning Rate	SP Val Loss	μ P Val Loss
1×10^{-5}	3.058	3.084
3×10^{-5}	2.245	2.235
1×10^{-4}	1.856	1.762
3×10^{-4}	1.525	1.505
1×10^{-3}	1.327	1.348
3×10^{-3}	1.255	1.249
1×10^{-2}	1.221*	1.225*

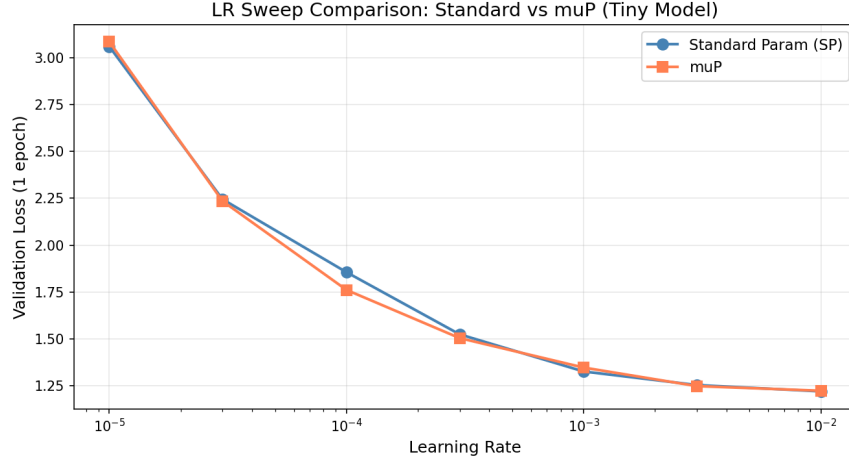


Figure 5: LR sweep comparison on the Tiny model. Both SP and μ P select 10^{-2} as the optimal LR. The μ P advantage emerges only when this LR is transferred to wider models.

5.3 SP vs. μ P Scaling Comparison

Table 6 and Figure 6 compare SP and μ P validation losses across model sizes. The μ P models have slightly higher parameter counts than their SP counterparts due to the absence of weight tying in MuReadout.

Table 6: Validation loss comparison: SP vs. μ P after 1 epoch. A positive “Improvement” value indicates μ P achieves lower loss.

Model	SP Params	SP Val Loss	μ P Val Loss	Improvement
Tiny	1.35M	1.229	1.221	+0.008
Small	3.51M	1.051	1.074	−0.023
Medium	12.32M	0.947	1.025	−0.078
Large	33.75M	1.067	0.941	+0.126
XL	88.40M	1.367	0.975	+0.392

The benefit of μ P increases markedly with model size. For the XL model, μ P reduces the validation loss by 0.392 (from 1.367 to 0.975), a substantial improvement. For Small and Medium models, SP slightly outperforms μ P, likely because the fixed LR happens to be near-optimal for those widths. The crossover point is between Medium and Large, consistent with the expected behavior that μ P becomes critical as the ratio of model width to base width grows.

The power law fit to the μ P results is:

$$L_{\mu P}(N) = 72852.3 \cdot N^{-0.866} + 0.952 \quad (R^2 = 0.960) \quad (5)$$

This fit is substantially better than the SP fit ($R^2 = 0.041$), reflecting the more regular scaling behavior under μ P. The scaling exponent $\alpha = 0.866$ indicates that every 10-fold increase in parameters reduces the loss by a factor of approximately $10^{0.866} \approx 7.3$ above the floor $c = 0.952$.

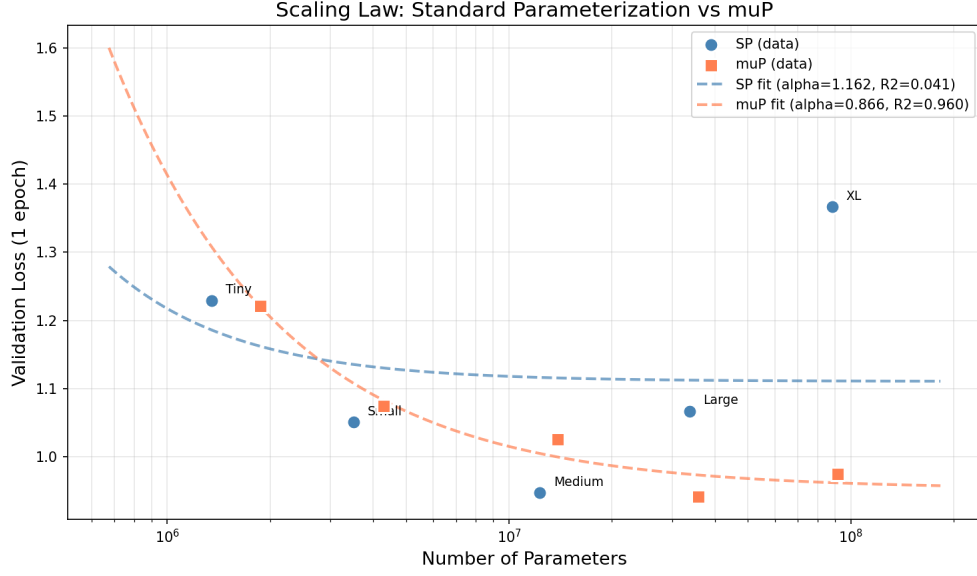


Figure 6: Scaling curves for SP (blue circles) and μ P (red squares) with power law fits. The SP curve is non-monotonic, yielding a degenerate fit ($R^2 = 0.041$). The μ P curve is smooth and yields a reliable fit ($R^2 = 0.960$, $\alpha = 0.866$).

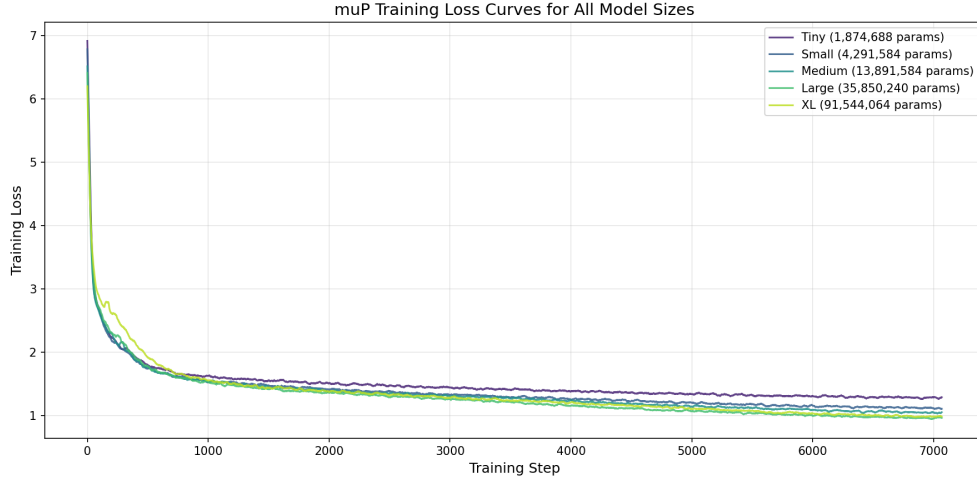


Figure 7: Smoothed training loss curves for all five μ P models. Unlike SP, the final training losses are roughly ordered by model size, consistent with the smoother validation loss scaling.

5.4 Scaling Law Extrapolation

Using the μ P power law fit (selected because $R^2 = 0.960 \gg 0.041$ for SP), the validation loss is extrapolated to a model with $10\times$ more parameters than the reference XL model (≈ 884 million parameters):

$$L_{\mu P}(8.84 \times 10^8) = 72852.3 \cdot (8.84 \times 10^8)^{-0.866} + 0.952 \approx 0.9535 \quad (6)$$

A 95% confidence interval is computed via error propagation using the covariance matrix returned by `scipy.optimize.curve_fit`:

$$\hat{L} = 0.954, \quad 95\% \text{ CI: } [0.888, 1.019], \quad \text{SE: } 0.033$$

Figure 8 shows the extrapolation with the confidence band.

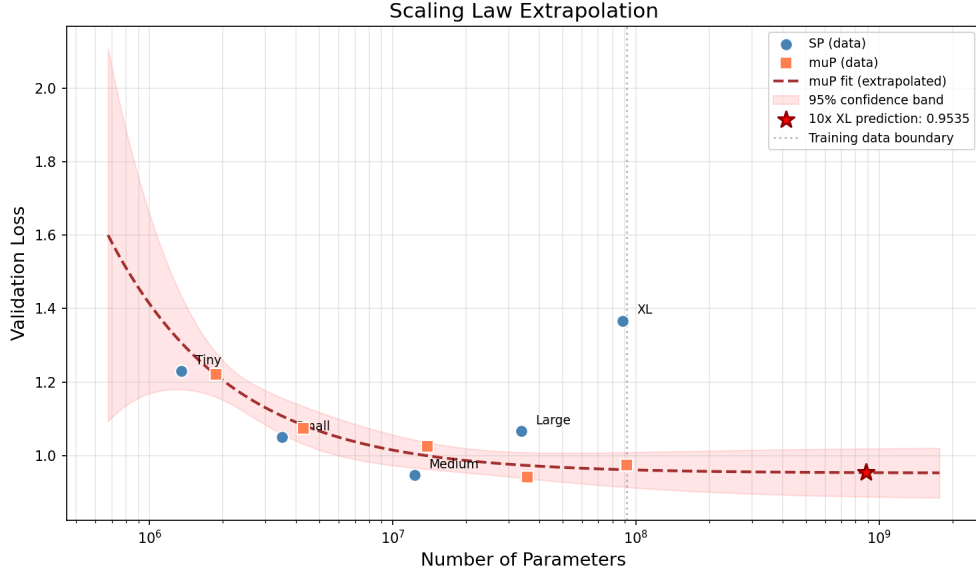


Figure 8: Scaling law extrapolation. The μP power law is extended to 8.84×10^8 parameters ($10\times$ the SP XL reference). The red star marks the predicted loss of 0.954. The shaded band shows the propagated 95% confidence interval. The vertical dotted line marks the boundary of the training data.

Uncertainty discussion: The confidence interval is narrow relative to the predicted value (half-width ≈ 0.065), but it reflects only statistical uncertainty from the fit to five data points. Practical uncertainty is much larger: (1) the power law may not hold a full order of magnitude beyond the training data; (2) dataset size and compute are held fixed while only N is extrapolated, departing from the Chinchilla-optimal regime [2]; (3) the loss floor $c = 0.952$ means further scaling returns diminishing improvements.

6 Best Model Training and SVG Generation

6.1 Model Selection and Extended Training

The XL model under μP is selected as the best model, motivated by its lowest one-epoch validation loss (0.975) among all configurations. It is trained for 5 epochs using the same optimizer, schedule, and batch size as the scaling study. The cosine schedule now spans all $5 \times 7,103 = 35,515$ steps with the learning rate decaying from 10^{-2} to 10^{-3} . Total training time is approximately 22,520 seconds (6.25 hours).

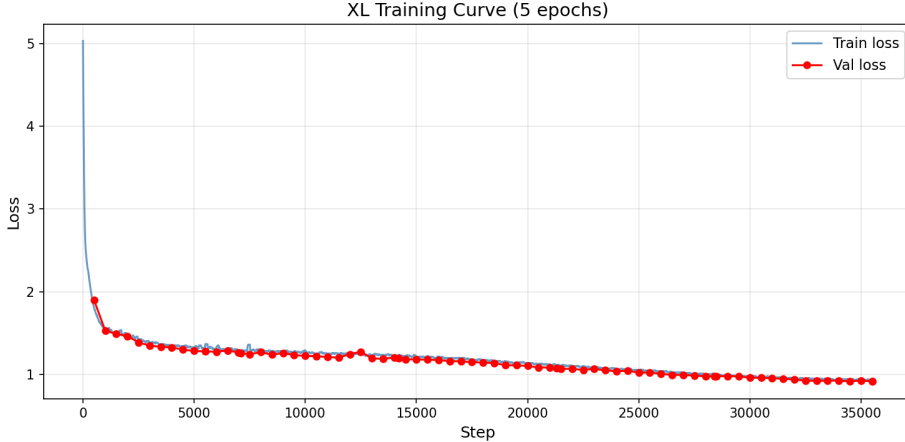


Figure 9: Training and validation loss curves for the XL μ P model trained for 5 epochs ($\approx 35,515$ steps). Validation loss reaches 0.920 at the end of training.

The final evaluation results on the held-out test set:

$$\mathcal{L}_{\text{test}} = 0.919, \quad \text{PPL} = \exp(0.919) \approx 2.51$$

6.2 Generation Setup

Samples are generated autoregressively from the `<svg ...>` prefix using top- k ($k = 50$) and nucleus (top- $p = 0.95$) sampling with temperature τ :

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (7)$$

A post-processing repair function handles truncated outputs: it removes incomplete tags, closes unclosed self-closing elements, balances `<g>` group tags, and appends `</svg>`.

6.3 Unconditional Generation

Eighteen unconditional samples are generated (6 per temperature: $\tau \in \{0.5, 0.8, 1.0\}$).

Table 7: Unconditional generation results by temperature.

Temperature	Samples	XML Valid	Renderable (strict)
0.5	6	5	1
0.8	6	4	2
1.0	6	5	5
Total	18	14	8

A counter-intuitive result is that temperature 1.0 produces the most renderable outputs (5/6), while temperature 0.5 produces the fewest (1/6). Inspection reveals that low-temperature sampling causes mode collapse: the model frequently predicts an end-of-sequence token immediately after the prefix,

producing 26-token degenerate outputs that contain only the opening `<svg>` tag followed by `</svg>`. At temperature 1.0, the model generates longer, more varied sequences with actual path and shape elements.

Figures 10 and 11 show rendered grids.

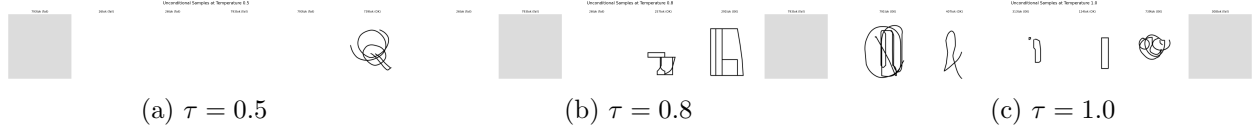


Figure 10: Unconditional generation grids at three temperatures. At $\tau = 0.5$, most outputs are degenerate (blank or near-blank) due to mode collapse. At $\tau = 1.0$, most outputs contain visible shapes.

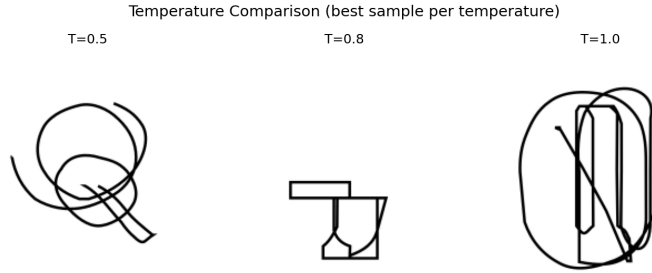


Figure 11: Best renderable sample from each temperature side by side.

6.4 Prefix-Conditioned Generation

Five prefix-conditioned samples are generated at $\tau = 0.8$, providing partial SVGs for the model to complete.

Table 8: Prefix-conditioned generation results.

ID	Description	New Tokens	XML Valid	Renderable (loose)
partial_face	Circle + eye; add other eye/mouth?	569	Yes	Yes
open_path	Open triangle; does model close it?	426	Yes	Yes
group_one_shape	Rectangle in group; add shapes?	1	Yes	Yes
star_start	Polygon start; complete the star?	768	Yes	Yes [†]
two_circles	Two circles; build a pattern?	374	Yes	Yes

All 5 prefix samples produce valid XML and are renderable under the CairoSVG loose check. The **star_start** case ([†]) hits the 768-token generation limit; CairoSVG processes the file successfully but the rendered output is a blank canvas (no visible pixels), so it would fail the strict alpha-channel check. The **group_one_shape** prefix generates only 1 new token, indicating the model immediately outputs a group-closing token rather than adding elements.

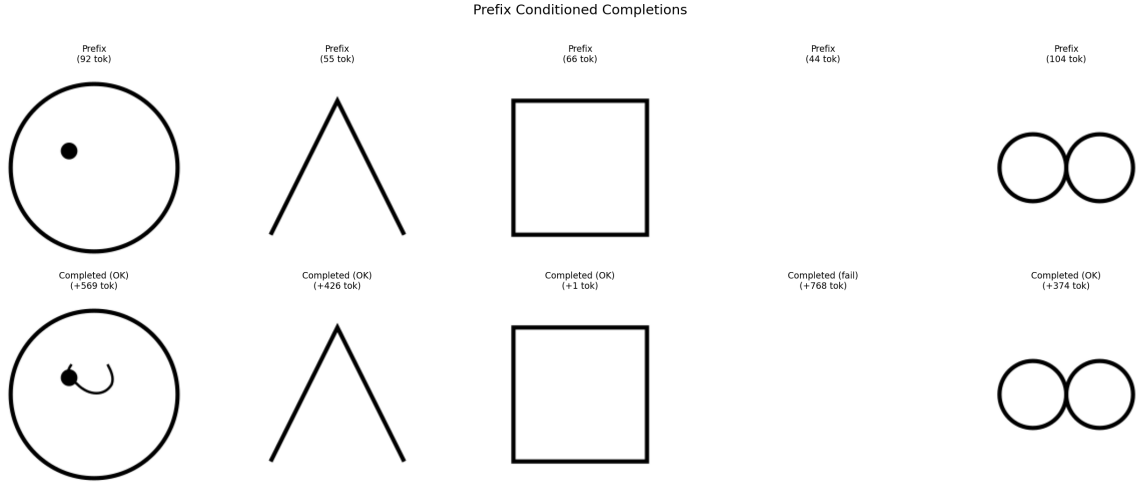


Figure 12: Prefix-conditioned completions. Top row: rendered prefix. Bottom row: model completion. The model generally produces geometrically coherent continuations.

6.5 Quantitative Evaluation Summary

Table 9: Quantitative evaluation of all 23 generated samples (18 unconditional + 5 prefix).

Metric	Value
Test perplexity	2.51
XML validity rate	82.6% (19/23)
SVG render rate (CairoSVG, loose)	82.6% (19/23)
SVG render rate (CairoSVG, strict)	52.2% (12/23)
Structural validity rate	82.6% (19/23)
viewBox usage	100% (23/23)
xmlns namespace present	100% (23/23)
Uses stroke attribute	70% (16/23)
Uses fill attribute	70% (16/23)
Uses <code><path></code> elements	65% (15/23)
Uses <code><g></code> groups	4% (1/23)

The model demonstrates strong mastery of SVG structural conventions: every generated sample includes the `xmlns` namespace declaration and a `viewBox` attribute, reflecting their near-universal presence in the training corpus. Path elements and basic attribute usage (stroke, fill) appear consistently. The low rate of group element usage (4%) likely reflects the training corpus composition: icon-style SVGs tend to use flat element hierarchies rather than nested groups.

7 Design Decisions and Analysis

Tokenization. A BPE vocabulary of 4,096 tokens balances subword granularity and embedding table size. Byte-level pre-tokenization ensures that no SVG character is out-of-vocabulary. Coordinate

rounding to one decimal place reduces the number of distinct numeric literals, improving token efficiency and reducing vocabulary fragmentation.

Context window. The block size of 256 tokens is shorter than the median SVG length (475 tokens), meaning the model sees only partial SVGs during training. A longer context window (512 or 1,024) would be preferable but was constrained by GPU memory for the largest models. This is a primary limitation: the model cannot maintain coherence over a full SVG during training.

Scaling insights. The SP scaling exponent ($\alpha = 1.16$) is unreliable due to the poor fit quality ($R^2 = 0.04$). The μ P exponent ($\alpha = 0.866$) is meaningful. This value is steeper than typical NLP exponents reported by Kaplan et al. [1] ($\alpha \approx 0.07$ for cross-entropy loss vs. compute), but those measurements optimize jointly over N , D , and C along the Pareto frontier. Our experiment holds D and batch size fixed and varies only N , so a direct comparison is not appropriate. Within these constraints, the observed exponent suggests that SVG syntax and local structure can be learned efficiently with more parameters, consistent with the highly structured nature of the domain.

Fixed LR vs. μ P. The crossover between SP and μ P occurs between the Medium (12M) and Large (33M) models. For models at or below Medium scale on this dataset, the fixed LR of 10^{-2} appears coincidentally near-optimal. For Large and XL models, the fixed LR is too large, causing the effective per-parameter update to grow with width under SP. μ P corrects this by scaling the per-layer LR inversely with width, stabilizing training for wider models.

Scale-dependent learning. Comparing Tiny (1.35M) and XL (91.5M) models under μ P reveals qualitative differences beyond the raw loss values. The Tiny model (val loss 1.221) generates syntactically plausible SVG tokens but frequently produces malformed attribute values and invalid coordinate sequences. The XL model (val loss 0.920 after 5 epochs) reliably produces valid XML structure, correct viewBox/xmlns headers, and geometrically coherent path commands. This suggests a rough division: smaller models learn token-level SVG syntax, while larger models internalize element-level constraints (attribute validity, command sequences). Global spatial coherence – symmetry, multiple aligned shapes – appears only partially even at XL scale, pointing to a third tier of structural understanding that likely requires either a longer context window or substantially more parameters.

SVG-specific patterns. All generated samples adopt the correct SVG header structure (xmlns, viewBox), which are also universal in the training set. The model consistently generates path (d=) attributes with valid move, line, and curve commands. However, group nesting and spatial symmetry (two eyes, balanced shapes) appear infrequently, suggesting that global compositional structure requires either more parameters or more training epochs than were feasible here.

Challenges. The primary challenge was the non-monotonic SP scaling, which necessitated μ P as a corrective intervention. The short context window (256 tokens) is a persistent limitation. The low-temperature mode collapse in generation was unexpected and reveals that the model’s token distribution is highly peaked around end-of-sequence tokens under low entropy, indicating the model may have been trained to terminate SVGs quickly in many examples.

8 Conclusion

This project demonstrates that power-law scaling laws can be empirically derived for SVG language models, but only when learning rates are properly transferred across model widths. Under standard parameterization with a fixed learning rate, the scaling behavior is non-monotonic and the power

law fit is essentially meaningless ($R^2 = 0.04$). μP restores reliable scaling ($R^2 = 0.960$, $\alpha = 0.866$) by eliminating the hyperparameter transfer problem. The best model (XL μP , 91.5M parameters, 5 epochs) achieves a test perplexity of 2.51 and generates SVGs that are valid XML in 82.6% of cases and exhibit strong mastery of SVG structural conventions. Limitations include the restricted context window (256 tokens vs. 475-token median SVG length), the absence of CairoSVG render validation in the preprocessing pipeline, and compute constraints that prevented training the largest model beyond 5 epochs or exploring models beyond 88M parameters.

References

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling Laws for Neural Language Models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, et al., “Training Compute-Optimal Large Language Models,” *arXiv preprint arXiv:2203.15556*, 2022.
- [3] G. Yang, E. J. Hu, I. Babuschkin, S. Sidor, X. Liu, D. Farhi, N. Ryder, J. Pachocki, W. Chen, and J. Gao, “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer,” *arXiv preprint arXiv:2203.09789*, 2022.
- [4] J. C. Rodriguez, P. Lezama, S. Mahendran, J. Delpino, J. Vargas, and G. Sapiro, “StarVector: Generating Scalable Vector Graphics Code from Images and Text,” *arXiv preprint arXiv:2312.11556*, 2023.
- [5] A. Karpathy, “nanoGPT,” <https://github.com/karpathy/nanoGPT>, 2022.
- [6] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems*, 32, 2019.
- [7] HuggingFace, “Tokenizers,” <https://github.com/huggingface/tokenizers>, 2020.
- [8] lxml Development Team, “lxml – XML and HTML with Python,” <https://lxml.de>, 2024.
- [9] CairoSVG Development Team, “CairoSVG,” <https://cairosvg.org>, 2024.

A Additional Generated Samples

Figures 13–15 show the full unconditional generation grids at each temperature. Generated SVG source files are included in the repository under `ml_final_project/outputs/generated/`.

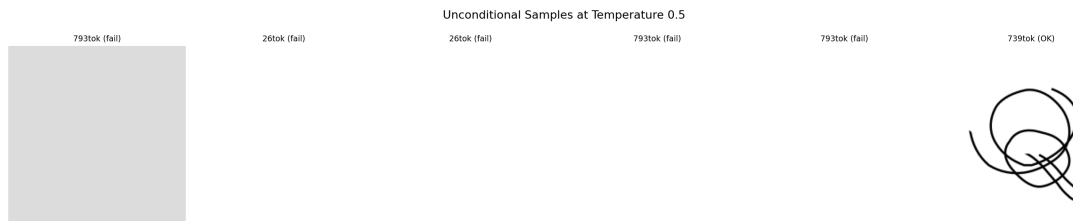


Figure 13: All unconditional samples at $\tau = 0.5$. Most outputs are degenerate.

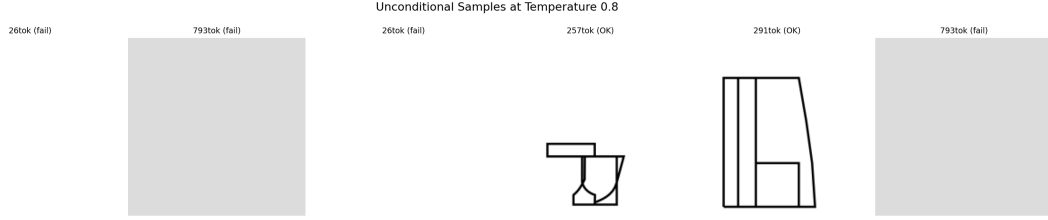


Figure 14: All unconditional samples at $\tau = 0.8$.

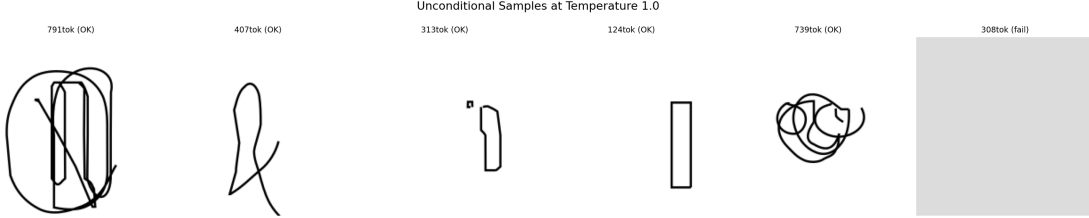


Figure 15: All unconditional samples at $\tau = 1.0$. Most outputs are renderable.

B Hyperparameter Summary

Table 10: Full hyperparameter summary for all experiments.

Hyperparameter	Value
Tokenizer	BPE, byte-level, vocab size 4096
Context window (block size)	256 tokens
Batch size	64 sequences (16,384 tokens/step)
Optimizer	AdamW
Adam betas	(0.9, 0.95)
Weight decay	0.1
Gradient clip norm	1.0
LR schedule	Cosine with 5% linear warmup
Min LR	$0.1 \times \text{max LR}$
Best LR (SP and μP)	10^{-2}
Dropout	0.1
μP base width	128
Part 4 epochs	5
Generation max tokens	768
Generation top- k	50
Generation top- p	0.95
Hardware	NVIDIA A100-SXM4-40GB

C Key Code Snippets

Listing 2: Power law model and fit (scipy).

```

1 def power_law(N, a, alpha, c):
2     return a * np.power(N, -alpha) + c
3
4 popt, pcov = curve_fit(
5     power_law, param_counts, val_losses,
6     p0=[10.0, 0.1, 1.0],
7     bounds=([0, 0, 0], [1e6, 2.0, 10.0]),
8     maxfev=10000,
9 )
10 a_fit, alpha_fit, c_fit = popt

```

Listing 3: Cosine LR schedule with linear warmup.

```

1 def get_lr(step, warmup_steps, max_steps, max_lr, min_lr):
2     if step < warmup_steps:
3         return max_lr * (step + 1) / warmup_steps
4     if step >= max_steps:
5         return min_lr
6     progress = (step - warmup_steps) / (max_steps - warmup_steps)
7     return min_lr + 0.5 * (max_lr - min_lr) * (1 + math.cos(math.pi *
    progress))

```

Listing 4: muP attention scaling ($1/d$ instead of $1/\sqrt{d}$).

```

1 # Standard parameterization:
2 att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(self.head_dim))
3
4 # muP parameterization:
5 att = (q @ k.transpose(-2, -1)) * (1.0 / self.head_dim)

```